Tentamen TEN1 HI1029 2025-03-10

Sista ordinarie insläpp: 8.00 – dörren stängs.
Tentavakterna delar ut konton. Logga in i tentasystemet så fort som möjligt.
När alla loggat in (förhoppningsvis 8.15) börjar tentamen och pågår i 4h 45min (tex 8.15-13.00).
8.30 Insläpp för försenade.

Extra skrivtid 50%: 7h 10min (tex 8.15-15.25) *(endast 1h extra skrivtid för A eller B)*
Skrivvakter: Anteckna start- och inlämningstid. Jour: Nicklas Brandefelt 087909738

Hjälpmedel: papper (tomma), penna, mus

Logga in med tentamenskontot ni får av skrivvakten. Logga inte ut förrän ni är klara med hela tentan.

Skapa ett nytt projekt i IntelliJ: **Project Name:** TEN1, **Project Location:** H

Skapa ett package per uppgift och kalla dessa uppgift1o2, uppgift3, uppgift4, uppgift 5, uppgift6. Kopiera hjälpfiler från P/HI1029/ till rätt package.

Svaren på teorifrågorna (fråga ett och två) skall besvaras i en textfil döpt till teori.txt som ni lägger i package uppgift1o2.

Även om ni inte löser hela uppgiften så kan ni få delpoäng men ni måste se till att koden kompilerar och kan köras. Både output och koden bedöms och det är viktigt att denna är välskriven och uppfyller uppgiftens instruktioner. Om ett program är ineffektivt kan detta ge poängavdrag. Det är meningen att man skall använda de tekniker vi lärt oss där detta är fördelaktigt. Välj bra namn för variabler, metoder och klasser. Se till att programmet har en bra struktur. Ni behöver på denna tentamen inte kommentera.

Hjälpfiler hittar ni i P/HI1029/ och Java API i P/Java/api/index.html.

Del 1 (15p): För godkänt krävs minst 1 poäng på teorifrågorna samt 9 poäng totalt. Om man får minst en poäng på teorifrågorna får man betyg enligt: 9p – E, 10p – D, 13p – C. Om man inte får minst en poäng på teorifrågorna men minst 8,5 poäng totalt får man Fx.

Del2: Bedöms bara om man får betyg C på del 1 och bedöms då tillsammans med del 1 med A, B eller C. Observera att om du har extra skrivtid får du bara använda 1 extra timme för att kunna få betyg A eller B.

Examinator och rättande lärare: Nicklas Brandefelt

Del 1

1. Förklara vad det betyder att en algoritm är $O(n^3)$. Ge exempel på minst två komplexitetsfunktioner som är $O(n^3)$ men som på andra sätt skiljer sig åt. Förklara hur två algoritmer som har dina två komplexitetsfunktioner skiljer sig åt och hur de påminner om varandra. (1p)

   Svaret skall lämnas i en textfil, teori.txt i package uppgift1o2.

2. Vad är en iterator? När och varför är det ibland viktigt att använda en sådan för att inte försämra effektiviteten? (1p)

   Svaret skall lämnas i en textfil, teori.txt i package uppgift1o2.

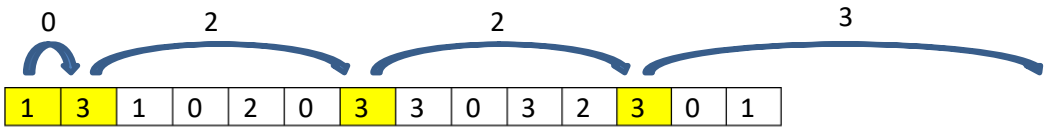
3. ***Ni som har fått godkänt på krysstalen i år hoppar över denna uppgift. Ange B på försättsbladet i rutan för uppgift 3.***
   Implementera en kö för data av typen String med hjälp av en enkellänkad lista som inre datastruktur. Du får inte använda någon datastruktur från Java's API (du får naturligtvis skapa klasser). Den ska förutom konstruktorn ha fyra publika metoderna enqueue (för att köa), dequeue (för att avköa), peak (för att titta på elementet först i kön) och isEmpty(). Alla metoder ska vara O(1). Om man använder dequeue eller peek på en tom kö skall dessa returnera null. (3p)

   En enkel main hittar ni i P/HI1029/uppgift3. Din kö ska gå att köra med denna. Fundera också på om det är några fler fall du behöver testa.

4. I en skolgårdslek ritar man en rad med rutor på asfalten. I rutorna lägger man noll till tre stenar. Ett av barnen ställer sig sedan i första rutan. Målet är att ha med sig så många stenar som möjligt när man har gått förbi hela raden. När man står i en ruta får man välja hur många stenar man tar upp ur rutan. Tar man upp 0 stenar går man en ruta fram men för varje sten man väljer att ta upp går man två rutor extra fram. Man får inte ta upp stenar från rutorna man passerar. Tar man t.ex. upp 2 stenar ska man alltså gå fem rutor fram. Där får man igen göra samma sak. Så fortsätter man tills man passerat sista rutan. Målet är att då ha så många stenar som möjligt.

   a) Du ska skriva ett program som tar en heltals-array som in-parameter och returnerar maximala antalet stenar man kan få med sig om arrayen innehåller antal stenar i varje ruta och index noll motsvarar första rutan.

   Array:en:



   ska alltså returnera 7.

   Enkla skal hittar du i P/HI1029/uppgift4. Du kan utgå ifrån att array:en endast innehåller talen 0, 1, 2 eller 3. Använd en rekursiv djupet först lösning. (2p)

   b) Effektivisera din lösning med hjälp av dynamisk programmering. Gör denna i en separat metod. (2p)

5. I den här uppgiften utgår vi ifrån vårt binära sökträd från föreläsningen (rensat). Utgå ifrån filerna i P/HI1029/uppgift5. Kom ihåg att metoderna ska kunna hantera även ett tomt träd. Försök skriva dem så effektiva som möjligt. Du ska inte lägga till några medlemsvariabler men får använda privata hjälpmetoder.

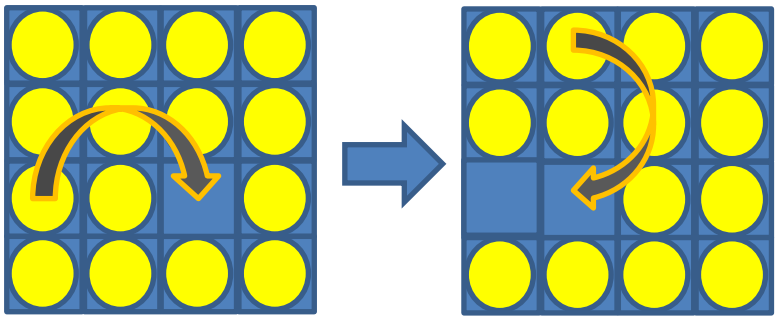
   a) Skriv add-metoden till trädet. Den ska returnera true om den lyckas annars false. (2p)

   b) Skriv en metod till trädet som returnerar true om trädet är ett perfekt binärt träd och annars false. (2p)

6. Skriv en metod som tar en sorterad array av heltal, och ett heltal: target som in-parameter. Den ska returnera antalet target i array:en och den ska göra detta med en bra söndra och härska algoritm. Om vi t.ex skickar in [2, 2, 2, 2, 4, 4, 4, 5, 7, 7, 7] och 4 ska den returnera 3 eftersom det finns tre fyror i array:en. Ett enkelt skal finns i P/HI1029/uppgift6. (2p)

Del 2

7. Spelet solitär börjar med att alla platser förutom en innehåller en kula. I varje drag får en kula hoppa över en annan kula och landa på en tom plats. Den överhoppade kulan tas bort. Man kan bara hoppa horisontellt eller vertikalt, inte diagonalt. Målet är att hoppa på ett sådant sätt att det i slutet endast finns en kula kvar. Vi ska här använda ett kvadratiskt bräde med bredden fyra. Skriv en metod som tar rad och kolumn för den tomma platsen och returnerar en dragsekvens (sträng) som resulterar i en kula kvar. Om lösning saknas returnerar den ”X”. Ett drag anges enligt (rad,kolumn)->(rad,kolumn) (vi numrerar från 0).



Som startdrag finns i exemplet ovan två möjligheter. Ovan visas startdraget (2,0)->(2,2) och brädet efter draget är slutfört. Som nästa drag finns tre möjligheter t.ex (0,1)->(2,1). Startplatsen enligt bilden (2,2) har ingen lösning men startplats (0,1) har en möjlig lösning i dragsekvensen (0,3)->(0,1), (0,0)->(0,2), (2,0)->(0,0), (2,1)->(0,1), (1,3)->(1,1), (3,2)->(1,2), (3,3)->(1,3), (0,1)->(2,1), (0,2)->(2,2), (3,0)->(3,2), (3,2)->(1,2), (1,3)->(1,1), (2,1)->(0,1), (0,0)->(0,2).

Även en inte perfekt utförd lösning kan räcka för betyget A eftersom det kan vara svårt att hinna med om man ska göra allt perfekt. För betyget B kan man nöja sig med att metoden returnerar true eller false beroende på om det finns eller inte finns en lösning. Observera att din lösning som kan kodas för storleken 4×4 ska vara sådan att den i princip hade fungerat även för andra storlekar på brädet. Ett enkelt skal finns i P/HI1029/uppgift7.